

Apache Hadoop* Community Spotlight

Apache* HDFS*

Konstantin Shvachko, Project Management Committee member for the Apache Hadoop* framework and founder of AltoScale, demystifies the Apache* Hadoop* Distributed File System (Apache HDFS*) and talks about where software development is headed.

Apache HDFS: Distributed Storage for Vast Quantities of Data

At the most basic level, the Apache Hadoop Distributed File System—or Apache HDFS—is the primary distributed storage component used by applications under the Apache open-source project Hadoop. HDFS can also serve as a stand-alone distributed file system.

The Apache Hadoop framework includes a number of components that support distributed computing to solve big data problems. At the lowest layer of the stack is Hadoop Common—those utilities that support other Hadoop modules. The next layer is HDFS, which provides the base for other layers to use for their processing—for example, Apache MapReduce, the Apache HBase* database, the Apache Hive* data warehouse infrastructure, and the Apache Pig* data flow language.

In addition, HDFS provides an abstraction, an API that enables users to deploy other distributed file systems or storage systems such as the Amazon* Simple Storage Service (S3), the open-source Parallel Virtual File System (PVFS), the IBM* General Parallel File System* (IBM GPFS*), the Lustre* file system, and most recently, the MapR* Map-Reduce Ready Distributed File System.

A Working Model

From the user point of view, HDFS is a traditional file system with a hierarchy of files and directories. It is implemented by two services that run on a cluster of commodity servers:

- **A single NameNode** maintains the directory tree and manages the namespace and access to files by clients.
- **DataNodes** store and manage the data blocks as local files on servers throughout the rest of the cluster.

“Hadoop 2.0 will include important new optimizations and functionality in HDFS*, making this work one of the most valuable improvements we can make in the near future.”*

—Konstantin Shvachko

HDFS splits files into large blocks, which are replicated on multiple DataNodes. This is done for both redundancy and availability of file data. Metadata is held by the NameNode, which executes namespace operations—opening, closing, and renaming files, for example—and maps the blocks to the DataNodes. In the current design, a single NameNode keeps the entire namespace in memory (RAM) while the data is replicated and stored on the DataNodes. The DataNodes serve requests from clients and manage data block operations as prescribed by the NameNode, such as deletion and replication.

In this model, the namespace is decoupled from data for optimal access of the metadata. Although requests for metadata are usually short and infrequent, typical Hadoop queries generate a large number of them. Decoupling provides the capability to stream data from DataNodes without interfering with the metadata operations.

User applications access the file system using the HDFS client, a library that exports the HDFS interface. The HDFS client accesses the NameNode for metadata about a data file, including the locations of the file blocks, and then streams data to or from DataNodes as the metadata prescribes.

Reliable, Scalable Processing and Storage: Four Design Principles

The open-source community for HDFS focuses development efforts on four critical design principles:

Linear scalability. HDFS is designed to handle vast quantities of unstructured and loosely structured data. As the volume of data continues to grow, both compute and storage scalability become critically important. HDFS is designed to scale linearly: Simply add more nodes (or servers) to the cluster to accelerate data processing and increase storage capacity.

System reliability and availability. With so many hardware components deployed in large Hadoop clusters, drive failure is very common. For example, a drive is expected to fail once in three years on average. When you have 1,000 drives or more in a cluster, some drives are failing every day. HDFS is designed with built-in fault-tolerance and automatic recovery capabilities. When HDFS breaks data into blocks and stores replicas on other servers in the cluster, the file system is creating redundancy. In the case of drive failure, this redundancy means the data is still available, and the system can continue to process requests against it.

Move computation to data. In the past, storage and computation were separated. Data was pulled from the storage system, processed, and returned to storage. Hadoop moves computation to where the data resides. MapReduce can assign workloads to the

How Does Apache* HDFS* Handle Large, Unstructured Data Sets?

The Apache* Hadoop* Distributed File System (HDFS*) does not assume any structure in a data set other than its characteristic as a set of files. To HDFS, each file is simply a group of bytes, and file structure is implemented at the application level. Consequently, the Apache Hadoop* framework can handle any unstructured or loosely structured data from a variety of sources, including weblogs, tweets, business documents, DNA data, and high-energy-physics data (Large Hadron Collider), to name a few. Because HDFS processes data sequentially, it is especially useful when large data sets are analyzed as a whole rather than when attempting to pinpoint a particular element in the data set.

servers where the data is stored, which reduces the overhead of large data transfers that can affect throughput—so important when processing large quantities of data.

Sequential data processing. HDFS is optimized for batch processing, with a focus on the overall throughput rather than the individual latency of operations. Sequential processing is intended to avoid random I/Os, or seeks, which can substantially slow down the I/O operations.

HDFS Limitations

HDFS can store very large data sets reliably and stream those data sets to user applications. However, as data volumes grow, HDFS and other Hadoop components will become limited by their single-master design. Because the metadata is stored in RAM on a single NameNode, there is a limit to how many objects you can store in that memory.¹ In addition, large loads on the system have the potential to create a performance bottleneck by overwhelming the single NameNode.

The Apache Hadoop community is working to overcome these limits to scalability. One potential way to solve the problem is with “federation.” Federation is already in place in the code, although not yet part of a stable release. It can use up to 10 NameNodes on a cluster sharing the same pool of DataNodes. This functionality will be part of the next stable release: Hadoop 2.0.²

However, ultimately federation has its own limitations. With 10 NameNodes in a cluster, users see 10 different file systems, or volumes. If one of those volumes grows faster than others, that NameNode will become overloaded, with no easy way to offload the metadata to other NameNodes. Dynamic partitioning of the namespace is the answer to this and is the goal of the Giraffa file system³, a new Apache Extras* project. Giraffa uses the HBase database, another storage component of the Hadoop framework that is maturing rapidly, as a metadata store. Traditional HDFS DataNodes would continue to serve for data storage. In this way, both parts of the file system—metadata and the data itself—will be distributed systems. This is similar to what Google is doing with its Colossus file system.

¹ This article includes specific numbers related to limitations to the RAM and the number of objects you can store. Shvachko, Konstantin V. “HDFS Scalability: The Limits to Growth.” *Usenix ;Login:* 35, No. 2 (April 2010). <http://c59951.r51.cf2.rackcdn.com/5424-1908-shvachko.pdf>

² The current stable version is Hadoop 1.0.

³ For more information about the Giraffa file system, see the presentation video from Hadoop Summit 2012. Shvachko, Konstantin V., and Plamen Jeliazkov. “Dynamic Namespace Partitioning with the Giraffa File System.” Hadoop Summit 2012 (June 14, 2012). [youtube.com/watch?v=tRVLNm_HM3I&feature=youtu.be](https://www.youtube.com/watch?v=tRVLNm_HM3I&feature=youtu.be)

Next Steps for HDFS

The HDFS community is actively engaged in stabilizing the software for the release of Hadoop 2.0. Hadoop 2.0 will include important new optimizations and functionality, making this the highest development priority for the near future. Longer term, the focus for the next generation of Hadoop and HDFS will continue to be on the four design principles. For example, improving scalability will always be important, because data is growing and computational power continues to get better. In addition, the community will be focusing on increased availability and expanded capabilities for real-time processing.

One of the next promising frontiers for the Apache Hadoop framework is the potential for the merger of Hadoop with cloud computing and virtualization. Bringing these technologies together magnifies the power of all three to resolve two important problems: Increase CPU utilization and provide resource isolation on the cluster.

Is Apache* HDFS* POSIX Compliant?

The Apache* Hadoop* Distributed File System (Apache HDFS*) is not fully compliant with Portable Operating System Interface (POSIX), a family of standards for maintaining compatibility between operating systems. By not implementing the full set of standards, HDFS gains the increased performance for handling very large data sets.

HDFS is mostly POSIX compliant, however. It does not support full access time on directories, random writes, or hard links, although in the case of hard links, support for symbolic links (symlinks) works for most Hadoop applications.

This paper is derived from an interview with Konstantin Shvachko on August 24, 2012. For the full interview, [listen to the podcast](#).

For more information about the Apache HDFS project, visit <http://hadoop.apache.org>.

Share with Colleagues    

This paper is for informational purposes only. THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NONINFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION, OR SAMPLE. Intel disclaims all liability, including liability for infringement of any property rights, relating to use of this information. No license, express or implied, by estoppel or otherwise, to any intellectual property rights is granted herein.

Copyright © 2012 Intel Corporation. All rights reserved. Intel, the Intel logo, Intel Sponsors of Tomorrow, and the Intel Sponsors of Tomorrow logo are trademarks of Intel Corporation in the U.S. and other countries.

*Other names and brands may be claimed as the property of others.

